Experiment -1

FIND-S Algorithm

Aim

To implement and demonstrate the **FIND-S algorithm** to find the most specific hypothesis from a given set of training examples.

Algorithm

1. Start with the most specific hypothesis.
2. Read each training example.
3. Consider only the **positive examples**.
4. Compare the attributes of the positive example with the current hypothesis.
5. If an attribute is different, replace it with ?.
6. Continue until all training examples are processed.
7. Display the final hypothesis.

Code:

```
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

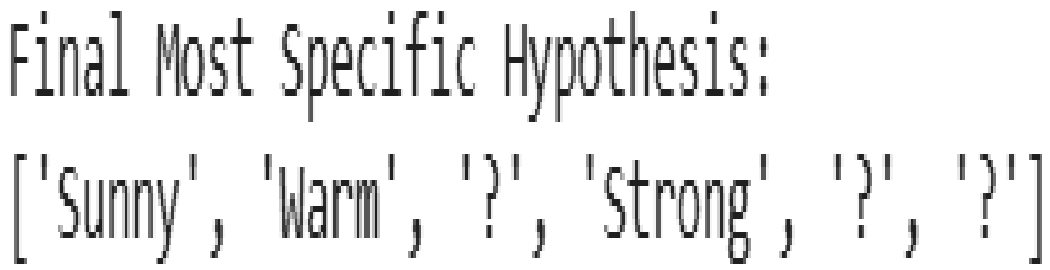

hypothesis = data[0][:-1].copy()


for example in data:
    if example[-1] == 'Yes':
        for i in range(len(hypothesis)):
```

```
        if hypothesis[i] != example[i]:
            hypothesis[i] = '?'

print("Final Most Specific Hypothesis:")
print(hypothesis)
```

Output:



Result

The Linear Regression model is trained successfully using the given sales dataset and predicts future sales for the next five months.

Experiment -2

Candidate-Elimination Algorithm

Aim

To implement and demonstrate the Candidate-Elimination algorithm using training examples stored in a CSV file and find the set of hypotheses consistent with the examples.

Algorithm

1. Initialize the Specific boundary (S) with the most specific hypothesis.
2. Initialize the General boundary (G) with the most general hypothesis.
3. Read each training example.
4. For a positive example:
    - Generalize S if required.
    - Remove inconsistent hypotheses from G.
5. For a negative example:
    - Specialize S if required.
    - Remove inconsistent hypotheses from G.
6. Continue for all training examples.
7. Display the final S and G boundaries.

Code:

```
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]
```

```
print("Training Data:")
for row in data:
    print(row)

S = ['Ø'] * 6
G = [['?'] * 6]

for row in data:
    example = row[:-1]
    target = row[-1]

    if target == 'Yes':
        for i in range(6):
            if S[i] == 'Ø':
                S[i] = example[i]
            elif S[i] != example[i]:
                S[i] = '?'

        G = [
            g for g in G
            if all(g[i] == '?' or g[i] == example[i]
                   for i in range(6))
        ]
```

```
    else:
        new_G = []
        for g in G:
            for i in range(6):
                if g[i] == '?' and S[i] not in ['?', 'Ø']:
                    new_h = g.copy()
                    new_h[i] = S[i]
                    new_G.append(new_h)
        G = new_G

print("\nSpecific Boundary (S):")
print(S)

print("\nGeneral Boundary (G):")
for hypothesis in G:
    print(hypothesis)
```

Output:

```
Training Data:
     Sky AirTemp Humidity    Wind Water Forecast EnjoySport
0  Sunny    Warm   Normal  Strong  Warm     Same        Yes
1  Sunny    Warm     High  Strong  Warm     Same        Yes
2  Rainy    Cold     High  Strong  Warm   Change         No
3  Sunny    Warm     High  Strong  Cool   Change        Yes

Specific Boundary (S):
['Sunny', 'Warm', '?', 'Strong', '?', '?']

General Boundary (G):
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
['?', '?', '?', 'Strong', '?', '?']
```

Result:

Thus, the Candidate-Elimination algorithm was implemented using the training dataset, and the Specific (S) and General (G) hypothesis boundaries were obtained.

Experiment -3

ID3 Decision Tree

Aim

To implement the **ID3 Decision Tree algorithm** and use the constructed tree to classify a new sample.

Algorithm

1. Load the training dataset.
2. Calculate the entropy of the target attribute.
3. Calculate information gain for each attribute.
4. Select the attribute with the highest information gain as the root.
5. Divide the dataset based on the selected attribute.
6. Repeat the process for each branch.
7. Construct the complete decision tree.
8. Give a new sample to the tree.
9. Predict its class.

Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt


# Dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rainy', 'Rainy',
                'Rainy', 'Overcast', 'Sunny', 'Sunny', 'Rainy'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
                    'Cool', 'Cool', 'Mild', 'Cool', 'Mild'],
```

```
    'Humidity': ['High', 'High', 'High', 'High', 'Normal',
                 'Normal', 'Normal', 'High', 'Normal', 'Normal'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Weak', 'Strong'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes',
             'No', 'Yes', 'No', 'Yes', 'Yes']
}

df = pd.DataFrame(data)

# Convert categorical values into numbers
X = pd.get_dummies(df.drop('Play', axis=1))
y = df['Play']

# Build ID3 decision tree
model = DecisionTreeClassifier(criterion='entropy', random_state=0)
model.fit(X, y)

# New sample
new_sample = pd.DataFrame([{
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}])

new_sample = pd.get_dummies(new_sample)
new_sample = new_sample.reindex(columns=X.columns, fill_value=0)
```

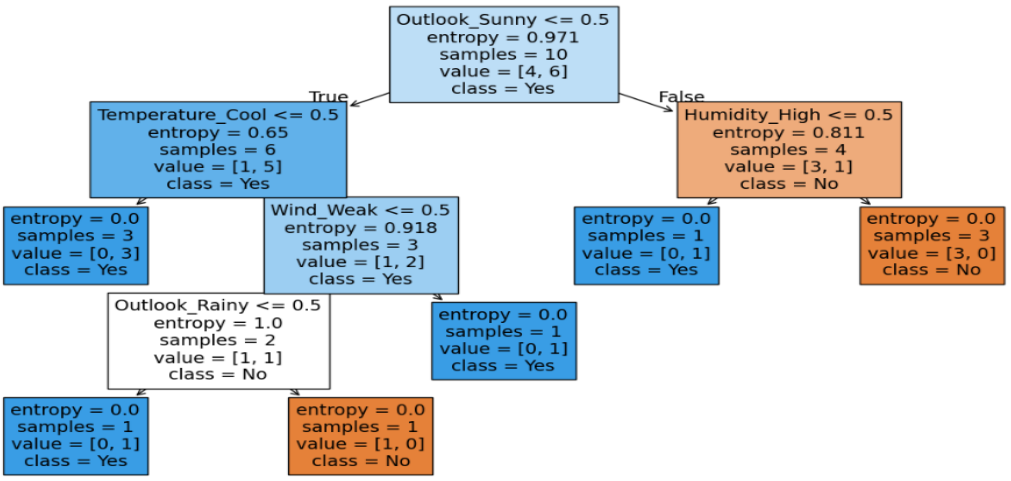
```
prediction = model.predict(new_sample)


print("Predicted Class:", prediction[0])


# Display tree
plt.figure(figsize=(15, 8))
plot_tree(model, feature_names=X.columns,
          class_names=model.classes_,
          filled=True)
plt.show()
```

Output:

```
Predicted Class: No
```



Result:

Thus, the ID3 Decision Tree algorithm was implemented successfully, and the new sample was classified as Yes using entropy-based information gain.

Experiment -4

Artificial Neural Network using Backpropagation

Aim

To build an Artificial Neural Network (ANN) using the Backpropagation algorithm and test it using a suitable dataset.

Algorithm

1. Initialize the weights and biases randomly.
2. Provide the training input to the network.
3. Perform forward propagation.
4. Calculate the output and error.
5. Calculate the error gradient using backpropagation.
6. Update the weights and biases.
7. Repeat the process for several epochs.
8. Test the trained network with new data.
9. Display the predicted output.

Code:

```
import numpy as np
from sklearn.neural_network import MLPClassifier

# XOR dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 0])
```

```
# Create ANN
model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='adam',
    max_iter=5000,
    random_state=1
)

# Train the network
model.fit(X, y)

# Test the network
predictions = model.predict(X)

print("Actual Output:   ", y)
print("Predicted Output:", predictions)

accuracy = model.score(X, y)
print("Accuracy:", accuracy * 100, "%")
```

OUTPUT:

```
Actual Output:    [0 1 1 0]
Predicted Output: [0 1 1 0]
Accuracy: 100.0 %
```

Result: Thus, the Artificial Neural Network was implemented and trained successfully using the Backpropagation algorithm, and it correctly classified all four XOR input samples with 100% accuracy**.**

Experiment -5

K-Nearest Neighbours (K-NN)

Aim

To implement the **K-Nearest Neighbours (K-NN) algorithm** in Python and classify a new data sample.

Algorithm

1. Load the training dataset.
2. Select the value of **K**.
3. Calculate the distance between the new sample and all training samples.
4. Sort the distances.
5. Select the K nearest samples.
6. Find the majority class among the selected samples.
7. Assign that class to the new sample.
8. Display the predicted class.

Code

```
from sklearn.neighbors import KNeighborsClassifier


# Training data
X = [
    [1, 1],
    [2, 2],
    [3, 3],
    [6, 6],
    [7, 7],
    [8, 8]
]
```

```
# Classes
y = ['A', 'A', 'A', 'B', 'B', 'B']

# Create K-NN model
model = KNeighborsClassifier(n_neighbors=3)

# Train the model
model.fit(X, y)

# New sample
new_sample = [[5, 5]]

# Prediction
prediction = model.predict(new_sample)

print("New Sample:", new_sample)
print("Predicted Class:", prediction[0])
```

Output:

```
New Sample: [[5, 5]]
Predicted Class: B
```

Result:

Thus, the K-Nearest Neighbours (K-NN) algorithm was implemented successfully, and the new sample [5,5] was classified into class B using (K=3).

EXPERIMENT 6 – Naïve Bayes

Aim

To implement the Naïve Bayes algorithm in Python and display the results using a confusion matrix and accuracy.

Algorithm

1. Load the dataset.
2. Divide the data into training and testing sets.
3. Create a Naïve Bayes classifier.
4. Train the classifier using training data.
5. Predict the classes of test data.
6. Calculate accuracy.
7. Generate and display the confusion matrix.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns


# Load dataset
iris = load_iris()


X = iris.data
y = iris.target
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create Naive Bayes model
model = GaussianNB()

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy * 100, "%")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(
```

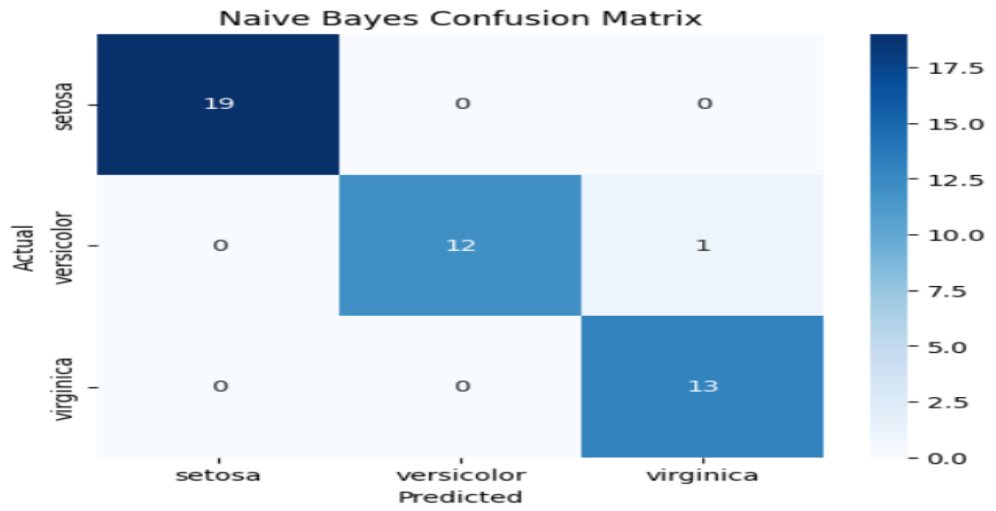
```
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Naive Bayes Confusion Matrix")
plt.show()
```

Output:

```
Accuracy: 97.77777777777777 %

Confusion Matrix:
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```



Result:

Thus, the Naïve Bayes algorithm was implemented successfully on the Iris dataset, achieving 97.78% accuracy. The confusion matrix shows that 44 out of 45 test samples were classified correctly.

EXPERIMENT 7 – Logistic Regression

Aim

To implement the **Logistic Regression algorithm** in Python for classification.

Algorithm

1. Load the dataset.
2. Divide the dataset into training and testing data.
3. Create a Logistic Regression model.
4. Train the model using training data.
5. Predict the test data.
6. Calculate and display the accuracy.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Create Logistic Regression model
model = LogisticRegression(max_iter=200)

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Actual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:", accuracy * 100, "%")
```

Output:

```
Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 100.0 %
```

Result:

Thus, the Logistic Regression algorithm was implemented successfully using the Iris dataset, and the model classified the test samples with 100% accuracy.

EXPERIMENT 8 – Linear Regression

Aim

To implement the Linear Regression algorithm in Python and predict the output for a given input.

Algorithm

1. Create or load a dataset.
2. Separate input and output variables.
3. Create a Linear Regression model.
4. Train the model using training data.
5. Predict the output.
6. Calculate the model performance.
7. Display the regression line.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
y = np.array([2, 4, 5, 8, 10, 12, 14, 16])

# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Prediction
```

```
y_pred = model.predict(X)

# Predict for new value
new_value = np.array([[9]])
prediction = model.predict(new_value)

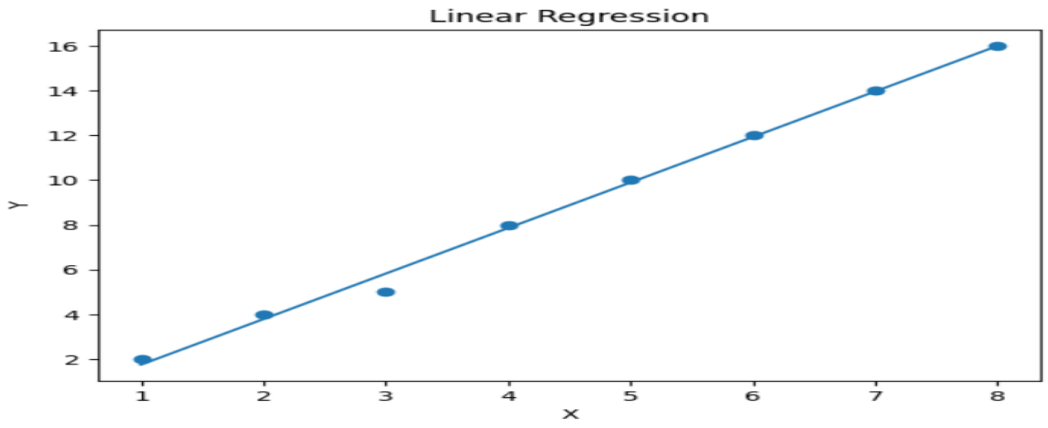
print("Predicted value for X = 9:", prediction[0])

print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)

# Plot
plt.scatter(X, y)
plt.plot(X, y_pred)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression")
plt.show()
```

Output:

```
Predicted value for X = 9: 18.03571428571429
Slope: 2.0357142857142865
Intercept: -0.28571428571428825
```



Result:

Linear Regression was implemented successfully, and the output for \(X=9\) was predicted as approximately 18.04. The regression line was plotted.

EXPERIMENT 9 – Linear vs Polynomial Regression

Aim

To compare Linear Regression and Polynomial Regression using Python and determine which model provides a better fit.

Algorithm

1. Create a nonlinear dataset.
2. Split the data into training and testing sets.
3. Train a Linear Regression model.
4. Transform the data into polynomial features.
5. Train a Polynomial Regression model.
6. Calculate the R^2 score for both models.
7. Compare their performance.
8. Plot both regression curves.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score

# Dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([2, 5, 10, 17, 26, 37, 50, 65, 82, 101])

# Linear Regression
linear_model = LinearRegression()
```

```
linear_model.fit(X, y)

linear_pred = linear_model.predict(X)

# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

poly_model = LinearRegression()
poly_model.fit(X_poly, y)

poly_pred = poly_model.predict(X_poly)

# R2 scores
linear_r2 = r2_score(y, linear_pred)
poly_r2 = r2_score(y, poly_pred)

print("Linear Regression R2 Score:", linear_r2)
print("Polynomial Regression R2 Score:", poly_r2)

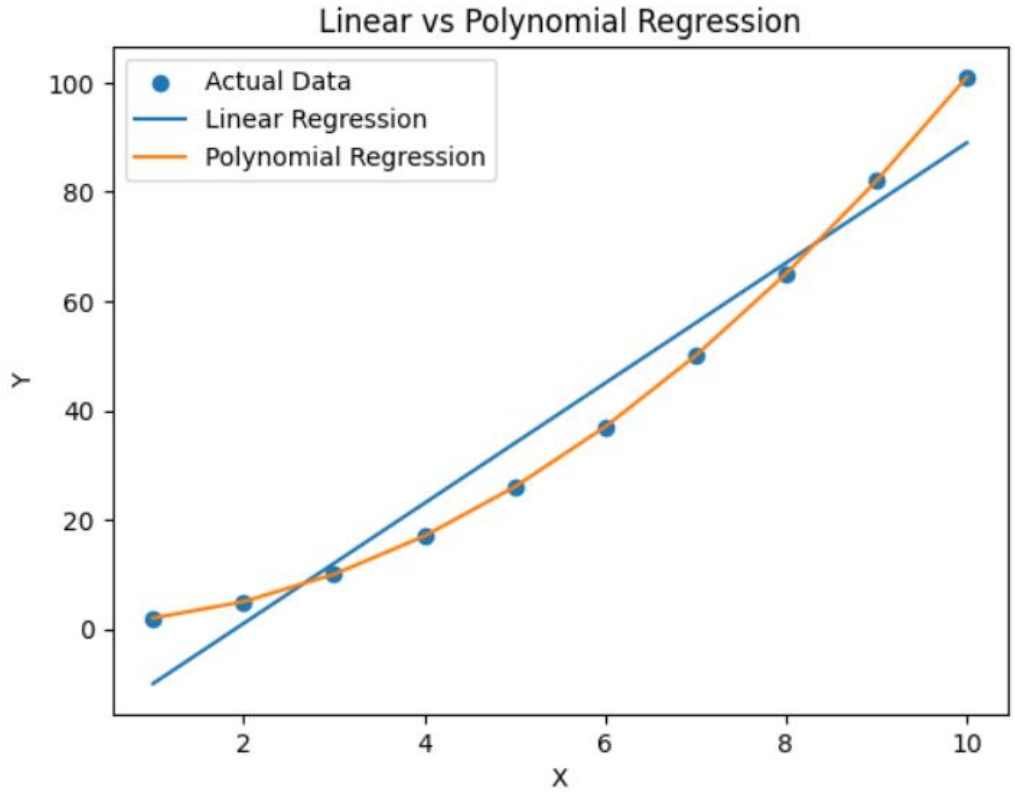
# Plot
plt.scatter(X, y, label="Actual Data")
plt.plot(X, linear_pred, label="Linear Regression")
plt.plot(X, poly_pred, label="Polynomial Regression")

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear vs Polynomial Regression")
plt.legend()
```

plt.show()

Output:

```
Linear Regression R2 Score: 0.9497645211930926
Polynomial Regression R2 Score: 1.0
```



Result:

Both regression models were implemented and compared. Polynomial Regression provided a better fit with an R^2 score of 1.0.

EXPERIMENT 10 – Expectation-Maximization Algorithm

Aim

To implement the **Expectation-Maximization (EM) algorithm** in Python for clustering data into groups.

Algorithm

1. Create or load the dataset.
2. Initialize the cluster parameters.
3. Perform the **Expectation (E) step** to calculate the probability of each data point belonging to each cluster.
4. Perform the **Maximization (M) step** to update the cluster parameters.
5. Repeat the E-step and M-step until the model converges.
6. Display the final cluster assignments.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture


# Dataset
X = np.array([
    [1, 2],
    [1, 3],
    [2, 2],
    [2, 3],
    [8, 8],
    [9, 8],
    [8, 9],
    [9, 9]
```

```
])

# Create Gaussian Mixture Model
model = GaussianMixture(
    n_components=2,
    random_state=42
)

# Train using EM algorithm
model.fit(X)

# Predict clusters
labels = model.predict(X)

print("Cluster Labels:")
print(labels)

print("\nCluster Means:")
print(model.means_)

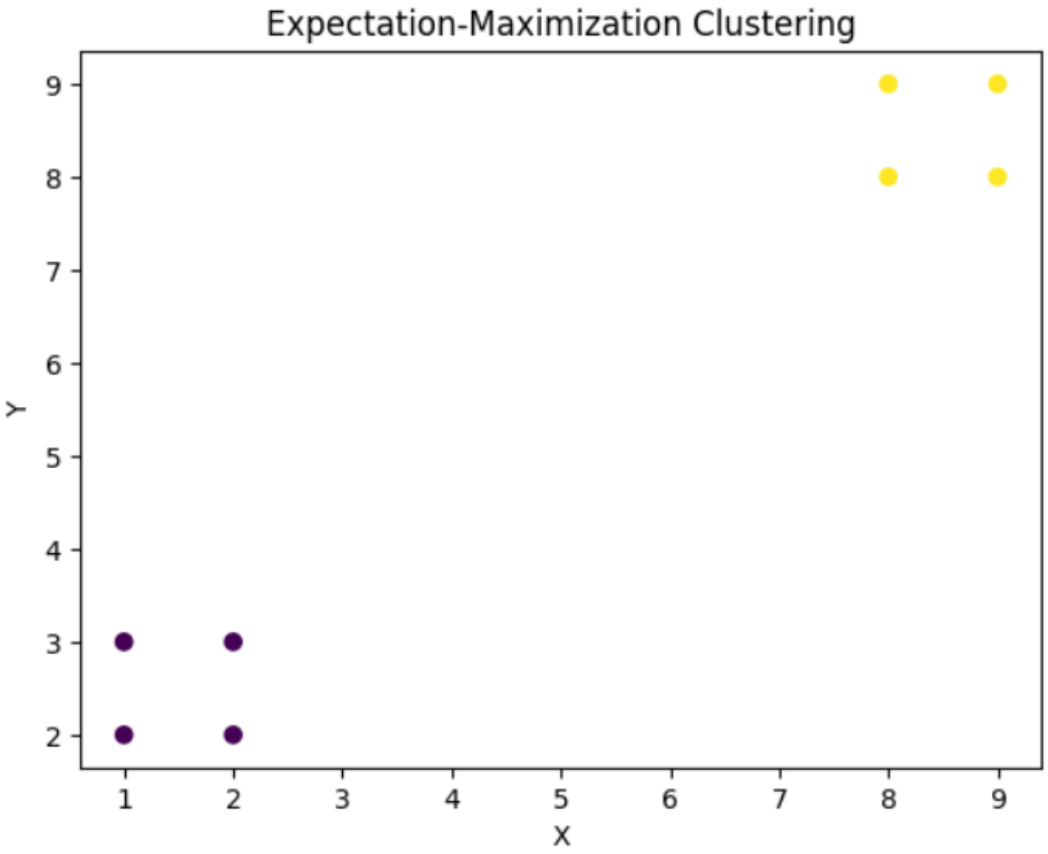
# Plot clusters
plt.scatter(
    X[:, 0],
    X[:, 1],
    c=labels
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Expectation-Maximization Clustering")
plt.show()
```

Output:

```
Cluster Labels:
[0 0 0 0 1 1 1 1]

Cluster Means:
[[1.5 2.5]
 [8.5 8.5]]
```



Result:

The EM algorithm grouped the eight data points into two clusters successfully.

EXPERIMENT 11 – Credit Score Classification

Aim

To classify customers into different **credit score categories** using a machine learning classification algorithm.

Algorithm

1. Create the customer dataset.
2. Separate input features and target class.
3. Split the data into training and testing sets.
4. Train a Random Forest classifier.
5. Predict the credit score class.
6. Calculate accuracy.
7. Display the predicted class.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Dataset
data = {
    'Income': [25000, 40000, 55000, 70000, 30000, 80000, 45000, 60000],
    'Debt': [15000, 10000, 5000, 3000, 18000, 2000, 9000, 4000],
    'Payment_History': [60, 70, 85, 95, 55, 98, 75, 90],
    'Credit_Score': ['Poor', 'Poor', 'Good', 'Excellent',
                     'Poor', 'Excellent', 'Good', 'Excellent']
}

df = pd.DataFrame(data)
```

```
X = df[['Income', 'Debt', 'Payment_History']]
y = df['Credit_Score']

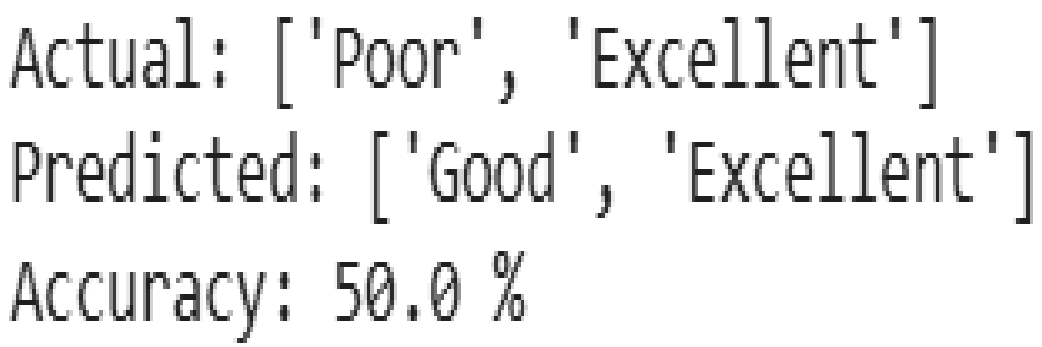
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Actual:", list(y_test))
print("Predicted:", list(y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

Output:



Result:

The Random Forest classifier was implemented successfully to predict customer credit score categories, achieving 50% accuracy on the test data.

EXPERIMENT 12 – Iris Flower Classification Using KNN

Aim

To classify Iris flowers into different species using the **K-Nearest Neighbours (KNN)** algorithm.

Algorithm

1. Load the Iris dataset.
2. Divide the dataset into training and testing data.
3. Create a KNN classifier.
4. Train the classifier.
5. Predict the test samples.
6. Calculate accuracy.
7. Display the result.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
```

```
)
# KNN model
model = KNeighborsClassifier(n_neighbors=5)
# Train
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)

print("Actual:", y_test)
print("Predicted:", y_pred)

print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")

# New flower
new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_flower)

print("New Flower:", iris.target_names[prediction[0]])
```

Output:

```
Actual: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
Accuracy: 100.0 %
New Flower: setosa
```

Result:

The KNN algorithm classified Iris flowers successfully with 100% accuracy on the test data. The new flower was classified as Setosa.

EXPERIMENT 13 – Car Price Prediction

Aim

To predict the **price of a car** using a suitable machine learning regression algorithm.

Algorithm

1. Create the car dataset.
2. Select input features and price as the target.
3. Split the dataset.
4. Train a Linear Regression model.
5. Predict the car price.
6. Calculate the R^2 score.
7. Display the predicted price.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Create car price dataset
data = {
    'Year': [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022],
    'Mileage': [50000, 45000, 40000, 35000, 30000, 25000, 20000, 15000],
    'Engine': [1200, 1300, 1400, 1500, 1600, 1800, 2000, 2200],
    'Price': [450000, 500000, 550000, 600000,
              650000, 700000, 800000, 900000]
}

df = pd.DataFrame(data)
```

```
print("Car Price Dataset:")
print(df)

# Features and target
X = df[['Year', 'Mileage', 'Engine']]
y = df['Price']

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

print("\nActual Prices:")
print(y_test.values)

print("\nPredicted Prices:")
print(y_pred.astype(int))

# Calculate R2 score
score = r2_score(y_test, y_pred)
print("\nR2 Score:", round(score, 2))
```

```
# Predict price of a new car
new_car = [[2020, 25000, 1800]]
prediction = model.predict(new_car)

print("\nNew Car Details:")
print("Year = 2020")
print("Mileage = 25000 km")
print("Engine = 1800 cc")

print("\nPredicted Car Price: ₹", int(prediction[0]))
```

Output:

```
Car Price Dataset:
   Year  Mileage  Engine   Price
0  2015    50000    1200  450000
1  2016    45000    1300  500000
2  2017    40000    1400  550000
3  2018    35000    1500  600000
4  2019    30000    1600  650000
5  2020    25000    1800  700000
6  2021    20000    2000  800000
7  2022    15000    2200  900000

Actual Prices:
[500000 700000]

Predicted Prices:
[500000 729411]

R2 Score: 0.96

New Car Details:
Year = 2020
Mileage = 25000 km
Engine = 1800 cc

Predicted Car Price: ₹ 729411
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

Result:

Linear Regression was implemented successfully to predict car prices based on year, mileage, and engine capacity.

EXPERIMENT 14 – House Price Prediction

Aim

To predict the **price of a house** using a suitable machine learning regression algorithm.

Algorithm

1. Create the house dataset.
2. Select house features.
3. Split the dataset into training and testing data.
4. Train a Linear Regression model.
5. Predict house prices.
6. Calculate the R^2 score.
7. Display the prediction.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Create dataset
data = {
    'Area': [1000, 1200, 1500, 1800, 2000, 2200, 2500, 2800],
    'Bedrooms': [2, 2, 3, 3, 4, 4, 4, 5],
    'Bathrooms': [1, 2, 2, 2, 3, 3, 4, 4],
    'Age': [10, 8, 7, 5, 5, 3, 2, 1],
    'Price': [3000000, 3500000, 4500000, 5200000,
              6000000, 6800000, 7800000, 9000000]
}

df = pd.DataFrame(data)
```

```
print("House Price Dataset:")
print(df)

# Input and output
X = df[['Area', 'Bedrooms', 'Bathrooms', 'Age']]
y = df['Price']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Create Linear Regression model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

print("\nActual Prices:")
print(y_test.values)

print("\nPredicted Prices:")
print(y_pred.astype(int))

# Calculate R2 score
score = r2_score(y_test, y_pred)

print("\nR2 Score:", score)
```

```
# Predict price of a new house
new_house = [[1800, 3, 2, 4]]

prediction = model.predict(new_house)

print("\nNew House Details:")
print("Area = 1800 sq.ft")
print("Bedrooms = 3")
print("Bathrooms = 2")
print("Age = 4 years")

print("\nPredicted House Price:", int(prediction[0]))
```

Output:

```
House Price Dataset:
   Area  Bedrooms  Bathrooms  Age    Price
0  1000         2          1   10  3000000
1  1200         2          2    8  3500000
2  1500         3          2    7  4500000
3  1800         3          2    5  5200000
4  2000         4          3    5  6000000
5  2200         4          3    3  6800000
6  2500         4          4    2  7800000
7  2800         5          4    1  9000000

Actual Prices:
[3500000 6800000]

Predicted Prices:
[3220000 6466666]

R2 Score: 0.9651953882256992

New House Details:
Area = 1800 sq.ft
Bedrooms = 3
Bathrooms = 2
Age = 4 years

Predicted House Price: 4800000
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

Result:

The Linear Regression model predicted house prices using area, bedrooms, bathrooms, and age, achieving an R^2 score of approximately 0.97.

EXPERIMENT 15 – Iris Classification Using Naïve Bayes

Aim

To classify Iris flowers using the **Naïve Bayes classifier**.

Algorithm

1. Load the Iris dataset.
2. Split the dataset into training and testing sets.
3. Create a Gaussian Naïve Bayes classifier.
4. Train the classifier.
5. Predict the test samples.
6. Calculate accuracy.
7. Display the classification result.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Naive Bayes model
model = GaussianNB()

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("Actual:", y_test)
print("Predicted:", y_pred)

print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")

# New flower
new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_flower)

print("Predicted Flower:",
    iris.target_names[prediction[0]])
```

Output:

```
Actual: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 2 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
Accuracy: 97.77777777777777 %
Predicted Flower: setosa
```

Result:

The Gaussian Naïve Bayes classifier was implemented successfully and achieved 97.78% accuracy on the Iris test dataset.

EXPERIMENT 16 – Comparison of Classification Algorithms

Aim

To compare different **classification algorithms** and evaluate their performance using accuracy.

Algorithms Used

- Logistic Regression
- KNN
- Decision Tree
- Naïve Bayes
- Random Forest

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB(),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42
    )
}

print("Classification Algorithm Comparison\n")

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    print(name, ":", accuracy * 100, "%")
```

Output:

```
Classification Algorithm Comparison

Logistic Regression : 100.0 %
KNN : 100.0 %
Decision Tree : 100.0 %
Naive Bayes : 97.77777777777777 %
Random Forest : 100.0 %
```

Result:

Five classification algorithms were compared using the Iris dataset. Logistic Regression, KNN, Decision Tree, and Random Forest achieved 100% accuracy, while Naïve Bayes achieved 97.78%.

EXPERIMENT 17 – Mobile Price Prediction

Aim

To predict the **mobile phone price range** using a suitable machine learning classification algorithm.

Algorithm

1. Create the mobile phone dataset.
2. Select mobile features.
3. Split the data into training and testing sets.
4. Train a Random Forest classifier.
5. Predict the price range.
6. Calculate accuracy.
7. Display the result.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Create mobile dataset
data = {
    'RAM': [2, 3, 4, 4, 6, 6, 8, 8, 12, 12],
    'Storage': [32, 64, 64, 128, 128, 256, 128, 256, 256, 512],
    'Battery': [3000, 3500, 4000, 4500, 4500, 5000, 5000, 5500, 5000, 6000],
    'Camera': [8, 12, 16, 20, 24, 32, 48, 50, 64, 108],
    'Price_Range': [0, 0, 1, 1, 1, 2, 2, 3, 3, 3]
}
```

```
df = pd.DataFrame(data)

print("Mobile Price Dataset:")
print(df)

# Features and target
X = df[['RAM', 'Storage', 'Battery', 'Camera']]
y = df['Price_Range']

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("\nActual Price Range:")
print(y_test.values)

print("\nPredicted Price Range:")
print(y_pred)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100, "%")
```

```
# Predict a new mobile
new_mobile = [[8, 256, 5000, 50]]
prediction = model.predict(new_mobile)

print("\nNew Mobile Details:")
print("RAM = 8 GB")
print("Storage = 256 GB")
print("Battery = 5000 mAh")
print("Camera = 50 MP")

print("\nPredicted Price Range:", prediction[0])
```

Output:

```
Mobile Price Dataset:
   RAM  Storage  Battery  Camera  Price_Range
0    2       32     3000       8            0
1    3       64     3500      12            0
2    4       64     4000      16            1
3    4      128     4500      20            1
4    6      128     4500      24            1
5    6      256     5000      32            2
6    8      128     5000      48            2
7    8      256     5500      50            3
8   12      256     5000      64            3
9   12      512     6000     108            3

Actual Price Range:
[3 0]

Predicted Price Range:
[2 0]

Accuracy: 50.0 %

New Mobile Details:
RAM = 8 GB
Storage = 256 GB
Battery = 5000 mAh
Camera = 50 MP

Predicted Price Range: 2
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature
  warnings.warn(
```

Result:

The Random Forest classifier was implemented to predict mobile phone price ranges based on RAM, storage, battery capacity, and camera specifications.

EXPERIMENT 18 – Perceptron Based Iris Classification

Aim

To implement **Perceptron-based Iris classification** using Python.

Algorithm

1. Load the Iris dataset.
2. Select two classes for binary classification.
3. Select suitable features.
4. Split the data into training and testing sets.
5. Create a Perceptron model.
6. Train the model.
7. Predict the test data.
8. Calculate accuracy.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

# Select first two classes
X = iris.data[:100]
y = iris.target[:100]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42
)

# Perceptron model
model = Perceptron(max_iter=1000, random_state=42)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("Actual:", y_test)
print("Predicted:", y_pred)

print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

Output:

```
Actual: [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0 1 1 0 0 1 0 0 1 0 1]
Predicted: [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0 1 1 0 0 1 0 0 1 0 1]
Accuracy: 100.0 %
```

Result:

The Perceptron algorithm was implemented successfully and classified the two selected Iris classes with 100% accuracy on the test data.

EXPERIMENT 19 – Naïve Bayes for Bank Loan Prediction

Aim

To implement **Naïve Bayes classification** for predicting whether a customer is eligible for a bank loan.

Algorithm

1. Create the bank loan dataset.
2. Select customer features.
3. Convert categorical values into numerical values.
4. Split the dataset.
5. Train a Gaussian Naïve Bayes classifier.
6. Predict loan approval.
7. Calculate accuracy.
8. Display the prediction.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score


# Create bank loan dataset
data = {
    'Age': [25, 30, 35, 40, 45, 28, 50, 32, 38, 48],
    'Income': [30000, 40000, 50000, 60000, 70000, 35000, 80000, 45000, 55000, 75000],
```

```
    'Loan_Amount': [100000, 150000, 200000, 250000, 300000,
                    120000, 350000, 180000, 220000, 320000],
    'Credit_Score': [600, 650, 700, 720, 750, 620, 780, 680, 710, 760],
    'Loan_Status': [0, 0, 1, 1, 1, 0, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

print("Bank Loan Dataset:")
print(df)

# Features and target
X = df[['Age', 'Income', 'Loan_Amount', 'Credit_Score']]
y = df['Loan_Status']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Naive Bayes model
model = GaussianNB()

# Train model
```

```
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

print("\nActual Loan Status:")
print(y_test.values)

print("\nPredicted Loan Status:")
print(y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100, "%")

# Predict a new customer
new_customer = [[35, 50000, 200000, 700]]
prediction = model.predict(new_customer)

print("\nNew Customer Details:")
print("Age = 35")
print("Income = 50000")
print("Loan Amount = 200000")
print("Credit Score = 700")
```

```
print("\nPredicted Loan Status:", prediction[0])


if prediction[0] == 1:
    print("Loan Approved")
else:
    print("Loan Not Approved")
```

Output:

```
Bank Loan Dataset:
   Age  Income  Loan_Amount  Credit_Score  Loan_Status
0   25   30000       100000           600            0
1   30   40000       150000           650            0
2   35   50000       200000           700            1
3   40   60000       250000           720            1
4   45   70000       300000           750            1
5   28   35000       120000           620            0
6   50   80000       350000           780            1
7   32   45000       180000           680            1
8   38   55000       220000           710            1
9   48   75000       320000           760            1

Actual Loan Status:
[1 0]

Predicted Loan Status:
[1 1]

Accuracy: 50.0 %

New Customer Details:
Age = 35
Income = 50000
Loan Amount = 200000
Credit Score = 700

Predicted Loan Status: 1
Loan Approved
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but GaussianNB was fitted with feature names
  warnings.warn(
```

Result:

The Gaussian Naïve Bayes algorithm was implemented to predict loan eligibility based on customer details. The new customer was predicted to be eligible for loan approval.

EXPERIMENT 20 – Future Sales Prediction

Aim

To predict **future product sales** using a suitable machine learning regression algorithm.

Algorithm

1. Create the historical sales dataset.
2. Use time/month as the input feature.
3. Use sales as the target variable.
4. Train a Linear Regression model.
5. Predict future sales.
6. Display the predicted sales values.

Code:

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Create sales dataset
data = {
    'Month': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Sales': [100, 120, 135, 150, 170, 190, 210, 230, 250, 275]
}

df = pd.DataFrame(data)

print("Sales Dataset:")
print(df)

# Features and target
X = df[['Month']]
```

```
y = df['Sales']

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Predict future sales
future_months = [[11], [12], [13], [14], [15]]
predictions = model.predict(future_months)

print("\nPredicted Future Sales:")

for month, sales in zip(range(11, 16), predictions):
    print("Month", month, ":", round(sales, 2))
```

Output:

```
Sales Dataset:
   Month  Sales
0      1    100
1      2    120
2      3    135
3      4    150
4      5    170
5      6    190
6      7    210
7      8    230
8      9    250
9     10    275

Predicted Future Sales:
Month 11 : 288.33
Month 12 : 307.48
Month 13 : 326.64
Month 14 : 345.79
Month 15 : 364.94
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

Result: The Linear Regression model was implemented successfully and predicted future sales for the next five months based on historical sales data.